

Application Layer: SMTP, DNS

CSE411

KyoungSoo Park

Department of Computer Science & Engineering
Seoul National University

Many slides are adapted from the publisher's material

All material copyright 1996-2020
J.F Kurose and K.W. Ross, All Rights Reserved

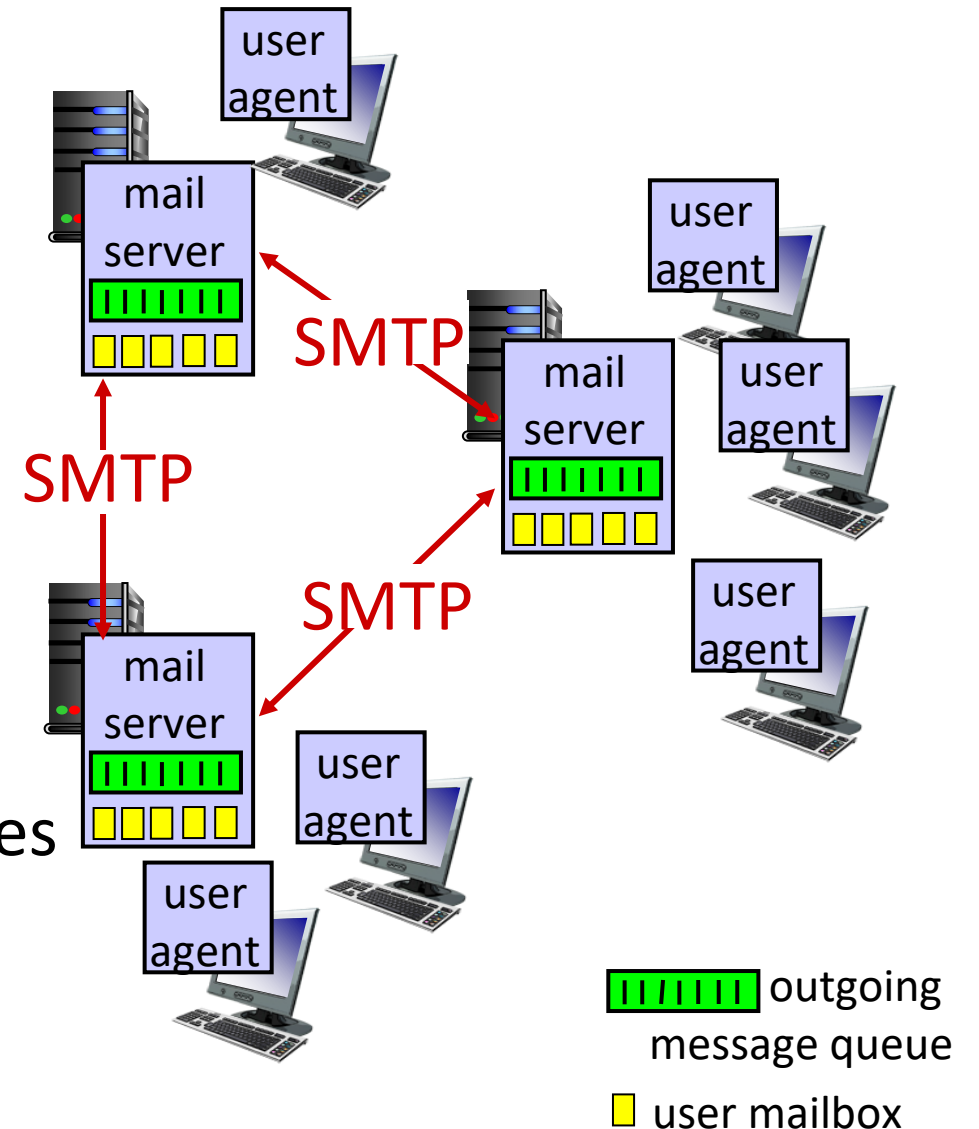
E-mail

Three major components:

- User agents
- Mail servers
- Simple mail transfer protocol: SMTP

User Agent

- a.k.a. “mail reader”
- Composing, editing, reading mail messages
- e.g., Outlook, iPhone mail client
- Outgoing, incoming messages stored on server



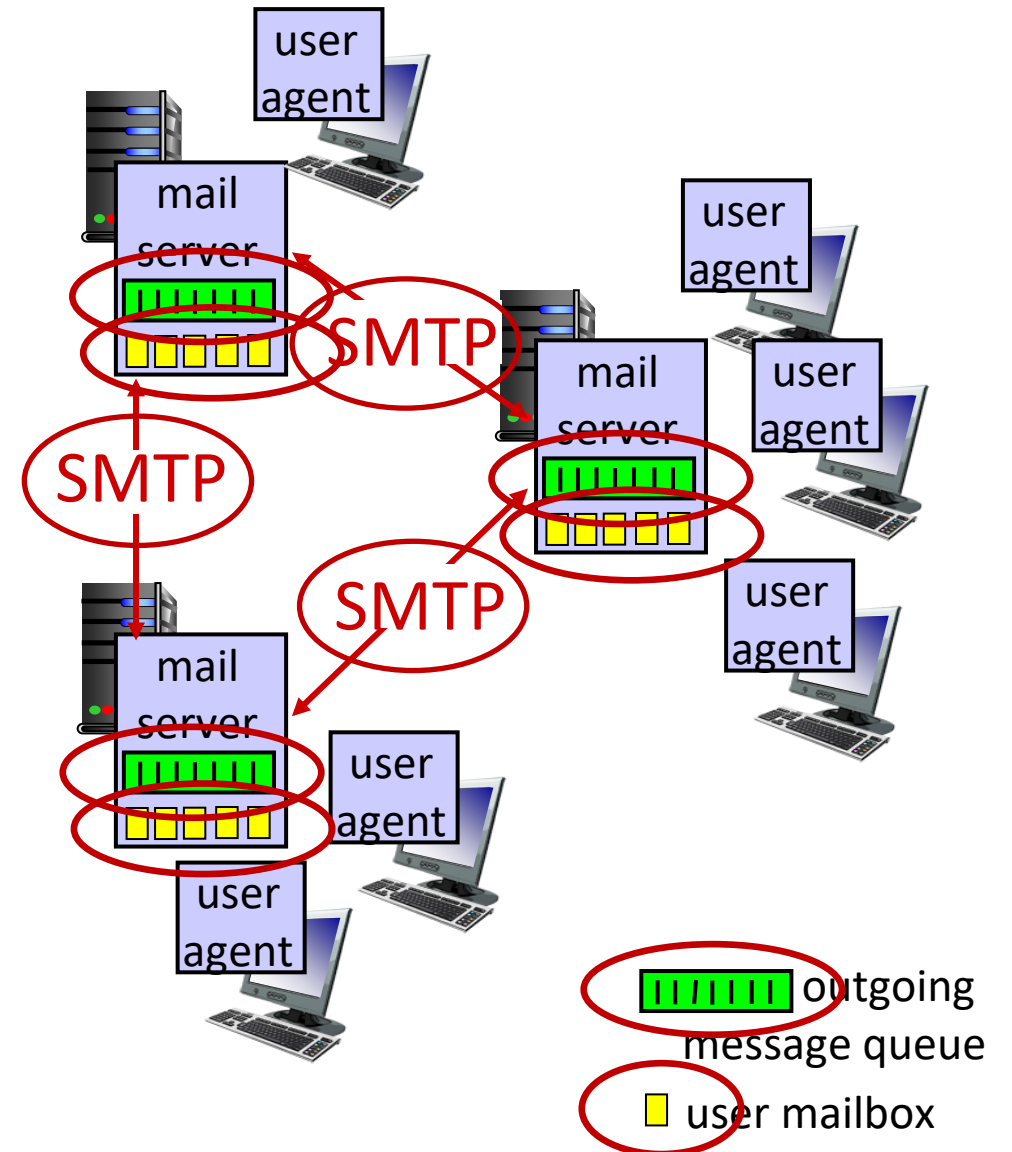
E-mail: mail servers

Mail servers:

- *Mailbox* contains incoming messages for user
- *Message queue* of outgoing (to be sent) mail messages

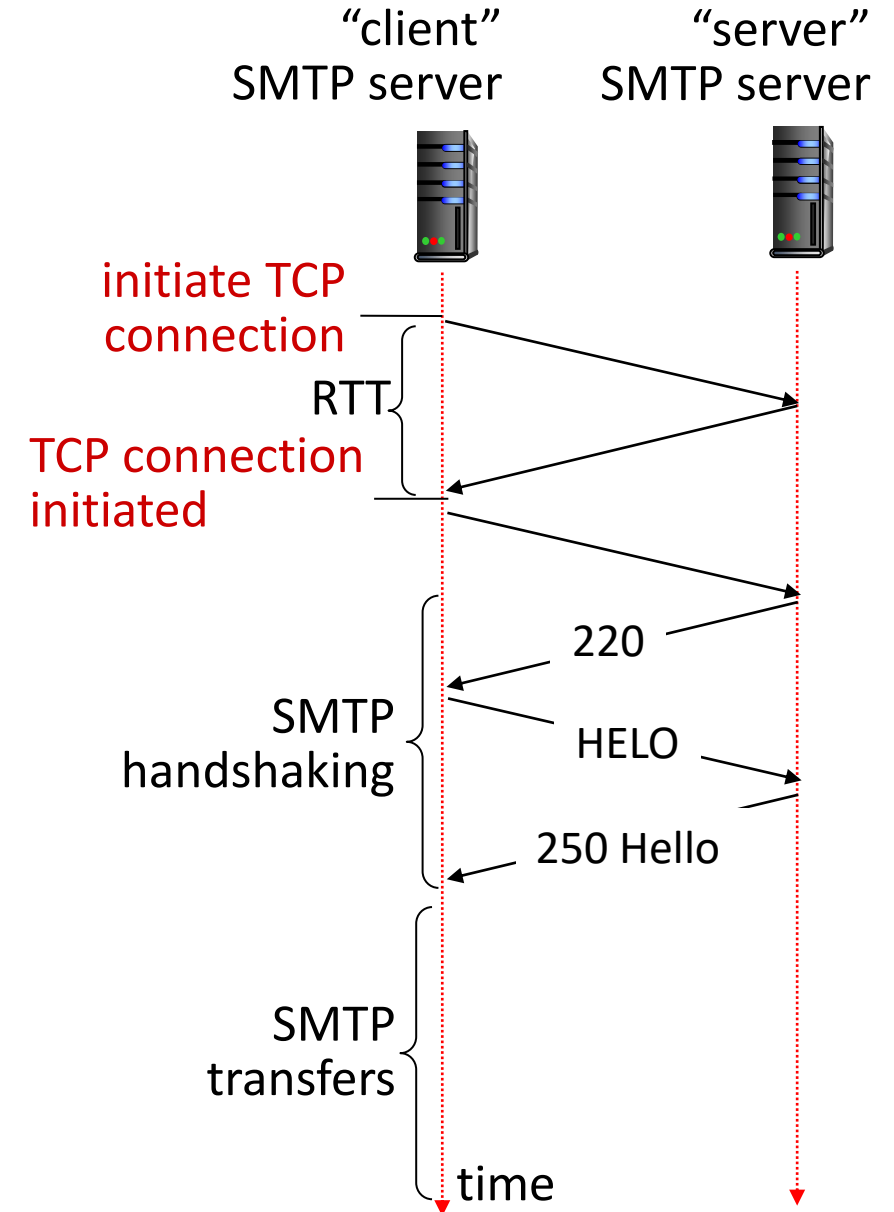
SMTP protocol between mail servers to send email messages

- **client**: sending mail server
- **server**: receiving mail server



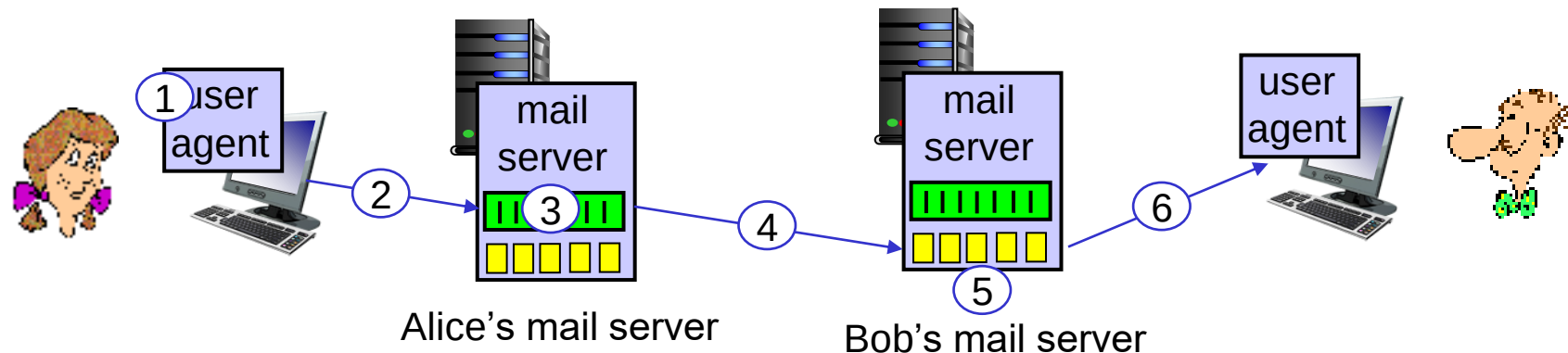
SMTP RFC (5321)

- Uses TCP to reliably transfer email message from client (mail server initiating connection) to server, port 25
 - Direct transfer: sending server (acting like client) to receiving server
- Three phases of transfer
 - SMTP handshaking (greeting)
 - SMTP transfer of messages
 - SMTP closure
- Command/response interaction (like HTTP)
 - **Commands:** ASCII text
 - **Response:** status code and phrase



Scenario: Alice sends e-mail to Bob

- 1) Alice uses UA to compose e-mail message "to" bob@someschool.edu
- 2) Alice's UA sends message to her mail server using SMTP; message placed in message queue
- 3) Client side of SMTP at mail server opens TCP connection with Bob's mail server
- 4) SMTP client sends Alice's message over the TCP connection
- 5) Bob's mail server places the message in Bob's mailbox
- 6) Bob invokes his user agent to read message



Sample SMTP interaction

dig mx syn.ac.kr!

S: 220 hamburger.edu

C: HELO crepes.fr

S: 250 Hello crepes.fr, pleased to meet you

C: MAIL FROM: <alice@crepes.fr>

S: 250 alice@crepes.fr... Sender ok

C: RCPT TO: <bob@hamburger.edu>

S: 250 bob@hamburger.edu ... Recipient ok

C: DATA

S: 354 Enter mail, end with "." on a line by itself

C: Do you like ketchup?

C: How about pickles?

C: .

S: 250 Message accepted for delivery

C: QUIT

S: 221 hamburger.edu closing connection

→ authentication required.

SMTP: observations

Comparison with HTTP:

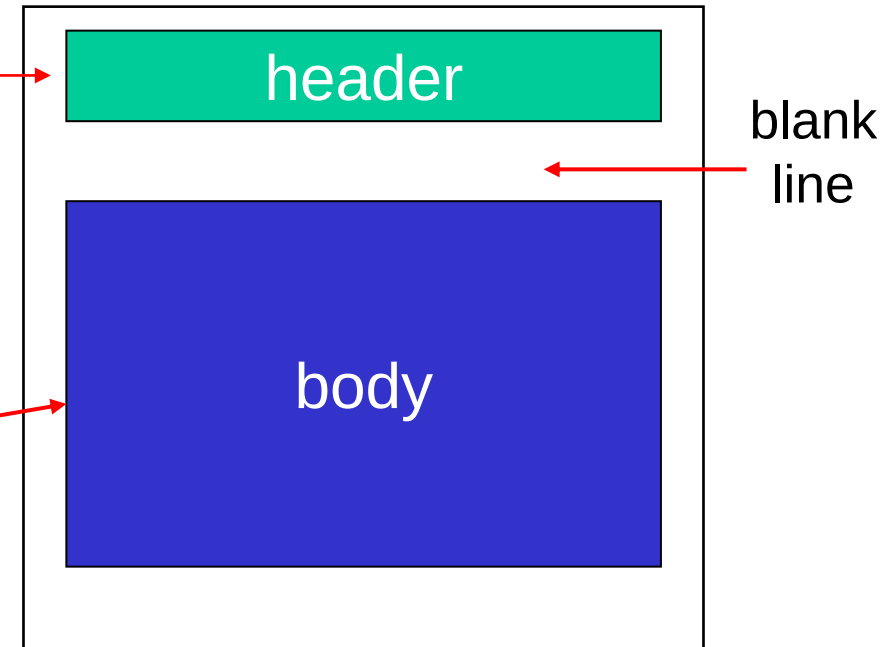
- HTTP: client pull
- SMTP: client push
- Both have ASCII command/response interaction, status codes
- HTTP: each object encapsulated in its own response message
- SMTP: multiple objects sent in multipart message
- SMTP uses persistent connections
- SMTP requires message (header & body) to be in 7-bit ASCII
- SMTP server uses CRLF.CRLF to determine end of message

Mail message format

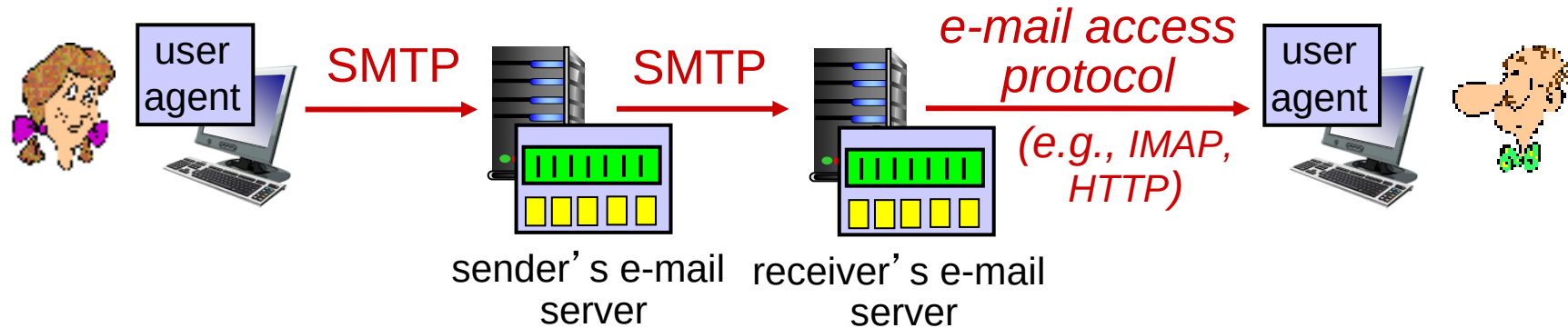
SMTP: protocol for exchanging e-mail messages, defined in RFC 5321 (like RFC 9110-9114 define HTTP)

RFC 2822 defines *syntax* for e-mail message itself (like HTML defines syntax for web documents)

- header lines, e.g.,
 - To:
 - From:
 - Subject:these lines, within the body of the email message area different from SMTP MAIL FROM:, RCPT TO: commands!
- Body: the “message” , ASCII characters only



Retrieving email: mail access protocols



- **SMTP:** delivery/storage of e-mail messages to receiver's server
- mail access protocol: retrieval from server
 - **IMAP:** Internet Mail Access Protocol [RFC 3501]: messages stored on server, IMAP provides retrieval, deletion, folders of stored messages on server
- **HTTP:** gmail, Hotmail, Yahoo!Mail, etc. provides web-based interface on top of SMTP (to send), IMAP (or POP) to retrieve e-mail messages

Domain Name System (DNS)

DNS: Domain Name System

use UDP.

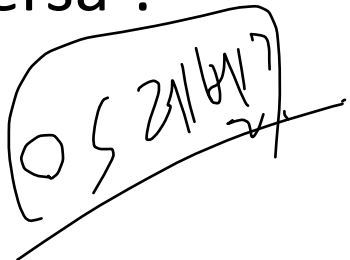
People: many identifiers:

- SSN, name, passport #

Internet hosts, routers:

- IP address (32 bit) - used for addressing datagrams
- “name”, e.g., cse.snu.ac.kr - used by humans

Q: how to map between IP address and name, and vice versa ?



Domain Name System (DNS):

- distributed database implemented in hierarchy of many name servers
- application-layer protocol: hosts, DNS servers communicate to resolve names (address/name translation)
 - note: core Internet function, implemented as application-layer protocol → usually not,
 - complexity at network's “edge”

core는 주로 계층/임계 수준에서
(router는 IP packet을 처리한다.)

DNS: services, structure

DNS services:

- Hostname-to-IP-address translation
- Host aliasing *vice versa*
 - Canonical, alias names
- Mail server aliasing
- Load distribution
 - Replicated Web servers: many IP addresses correspond to one name

Q: Why not centralize DNS?

- Single point of failure
- Traffic volume
- Distant centralized database
- Maintenance

A: Doesn't scale!

- Comcast DNS servers alone: 600B DNS queries/day
- Akamai DNS servers alone: 2.2T DNS queries/day

Thinking about the DNS

Humongous distributed database:

- ~ billion records, each simple

Handles many *trillions* of queries/day:

- *Many* more reads than writes
- *Performance matters*: almost every Internet transaction interacts with DNS - msec count!

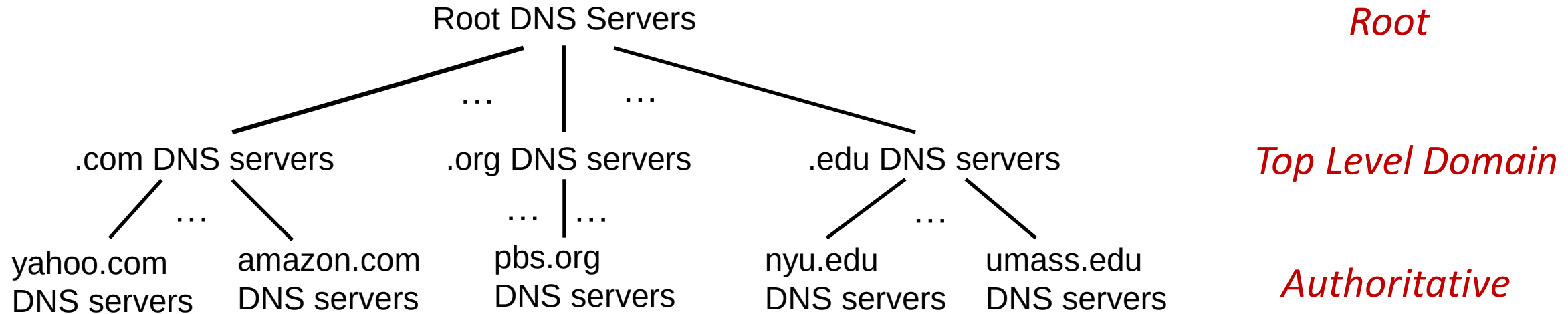
Organizationally, physically decentralized:

- Millions of different organizations responsible for their records

“Bulletproof”: reliability, security



DNS: a distributed, hierarchical database

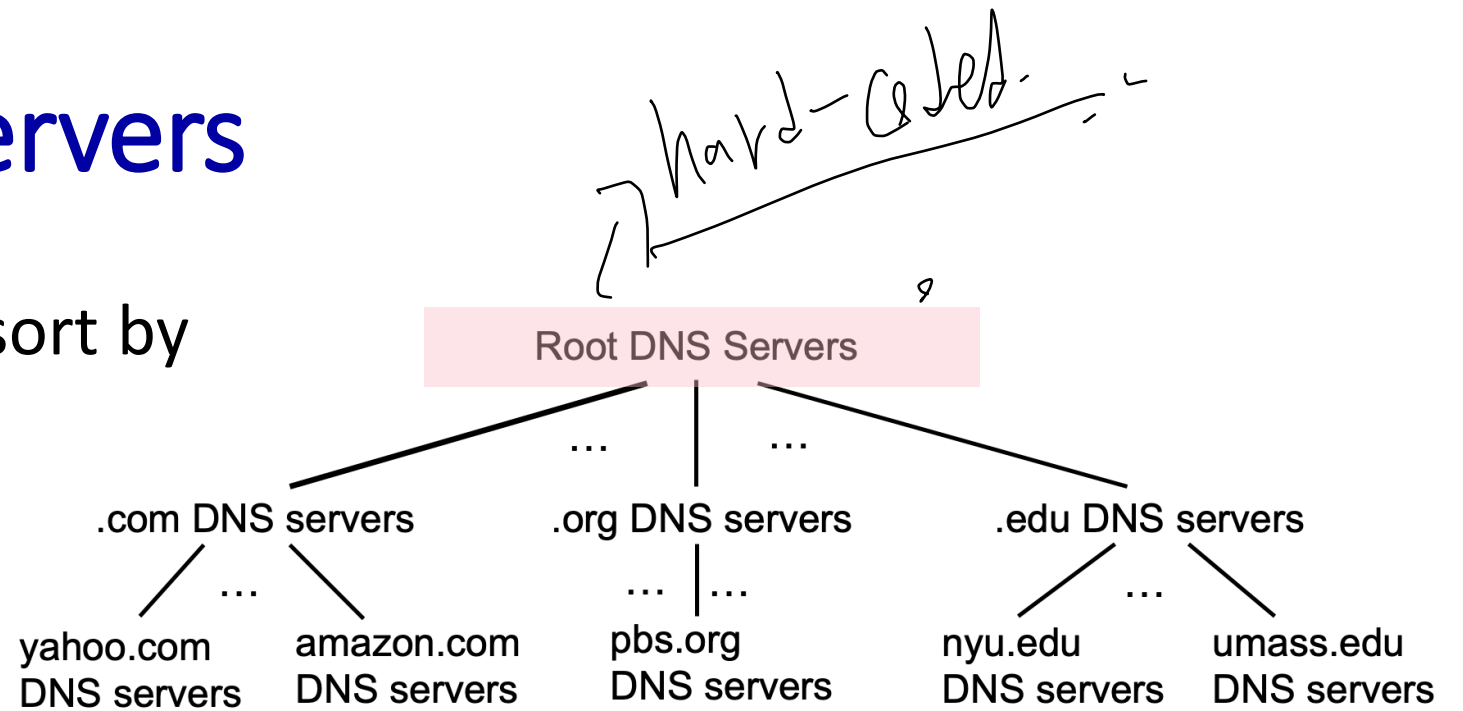


Client wants IP address for `www.amazon.com`; 1st approximation:

- Client queries root server to find `.com` DNS server
- Client queries `.com` DNS server to get `amazon.com` DNS server
- Client queries `amazon.com` DNS server to get IP address for `www.amazon.com`

DNS: root name servers

- Official, contact-of-last-resort by name servers that can not resolve name

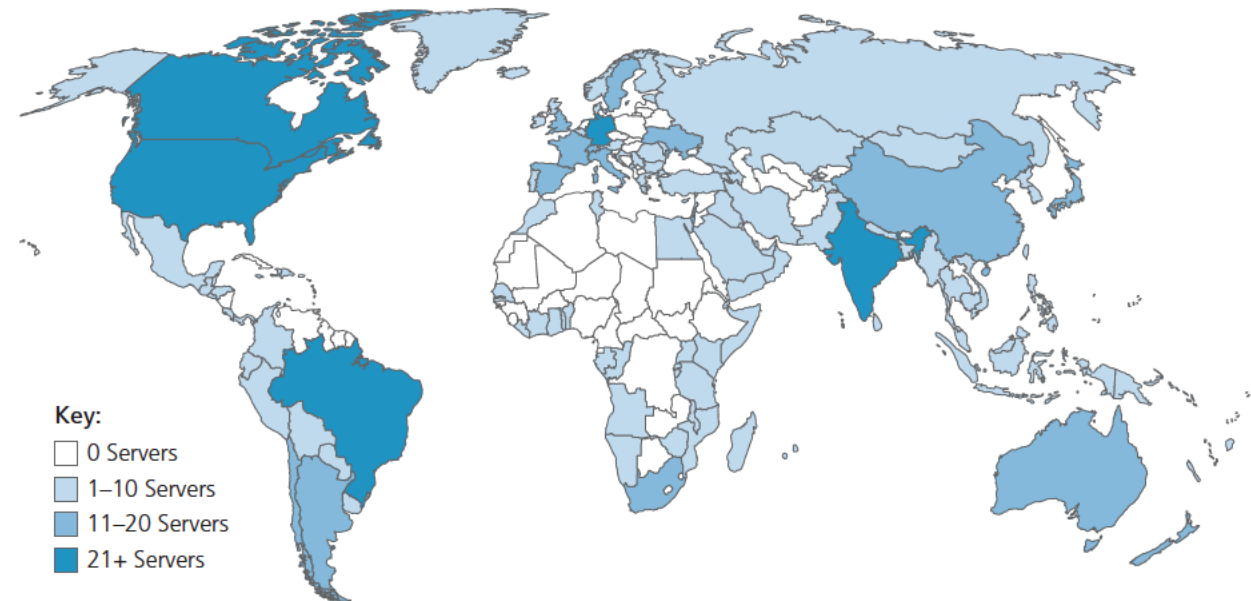


- Com is the root of the tree.

DNS: root name servers

- Official, contact-of-last-resort by name servers that can not resolve name
- *Incredibly important* Internet function
 - Internet couldn't function without it!
 - DNSSEC – provides security (authentication, message integrity)
- ICANN (Internet Corporation for Assigned Names and Numbers) manages root DNS domain

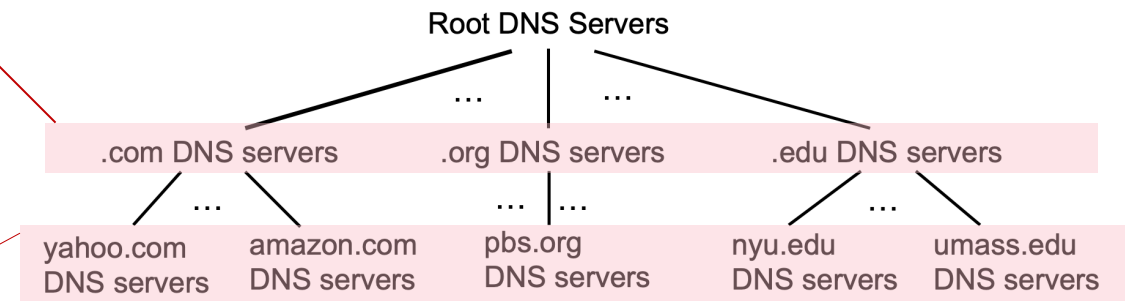
13 logical root name “servers”
worldwide each “server” replicated
many times (~200 servers in US)



Top-Level Domain, and authoritative servers

Top-Level Domain (TLD) servers:

- Responsible for .com, .org, .net, .edu, .aero, .jobs, .museums, and all top-level country domains, e.g.: .cn, .uk, .fr, .ca, .jp
- Network Solutions: authoritative registry for .com, .net TLD
- Educause: .edu TLD



Authoritative DNS servers:

- Organization's own DNS server(s), providing authoritative hostname to IP mappings for organization's named hosts
- Can be maintained by organization or service provider

Local DNS name servers

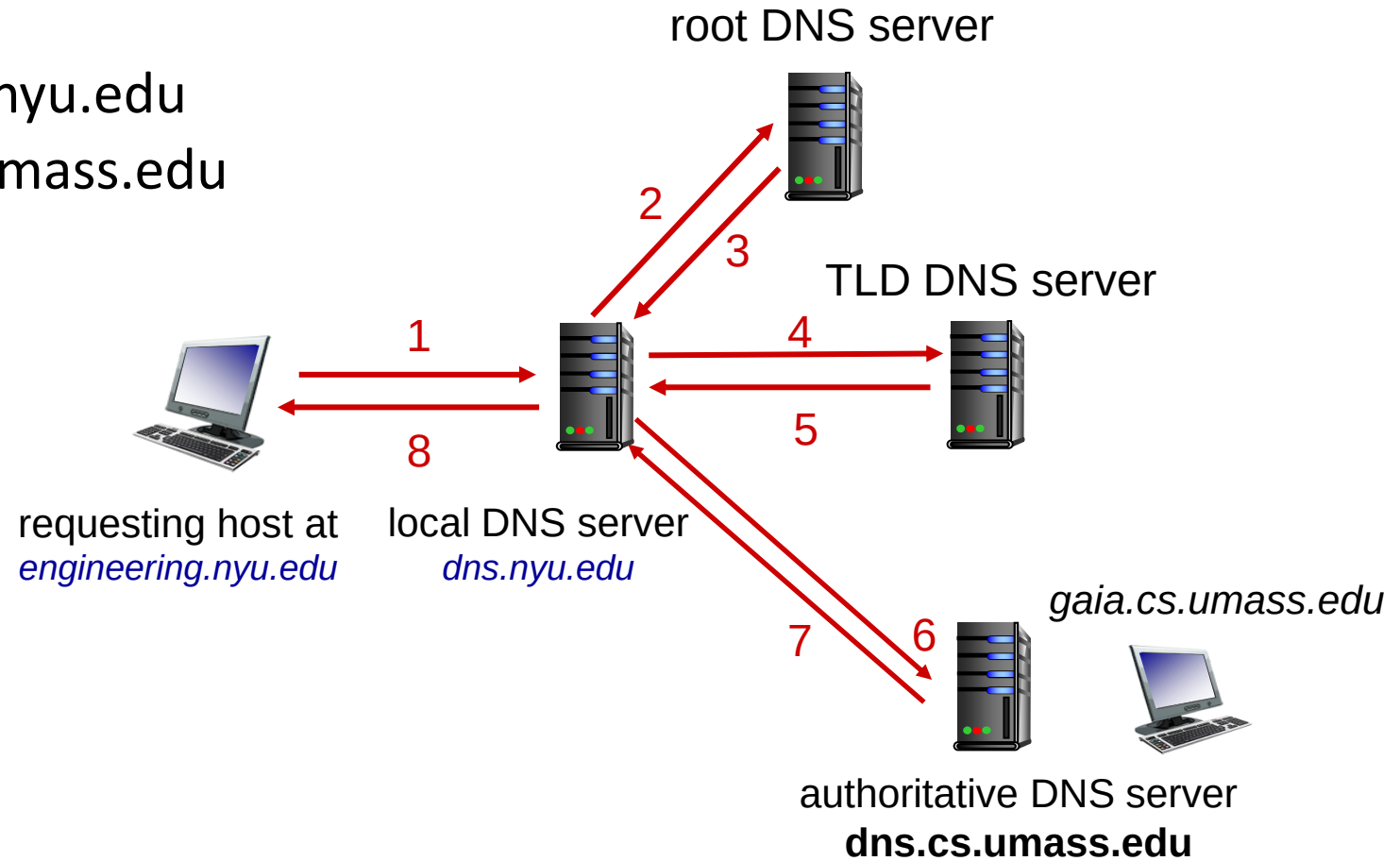
- When host makes DNS query, it is sent to its *local* DNS server
 - Local DNS server returns reply, answering:
 - From its local cache of recent name-to-address translation pairs (possibly out of date!)
 - Forwarding request into DNS hierarchy for resolution
 - Each ISP has local DNS name server; to find yours:
 - MacOS: `% scutil --dns`
 - Windows: `>ipconfig /all`
- Local DNS server doesn't strictly belong to hierarchy

DNS name resolution: iterated query

Example: host at `engineering.nyu.edu` wants IP address for `gaia.cs.umass.edu`

Iterated query:

- Contacted server replies with name of server to contact
- “I don’t know this name, but ask this server”

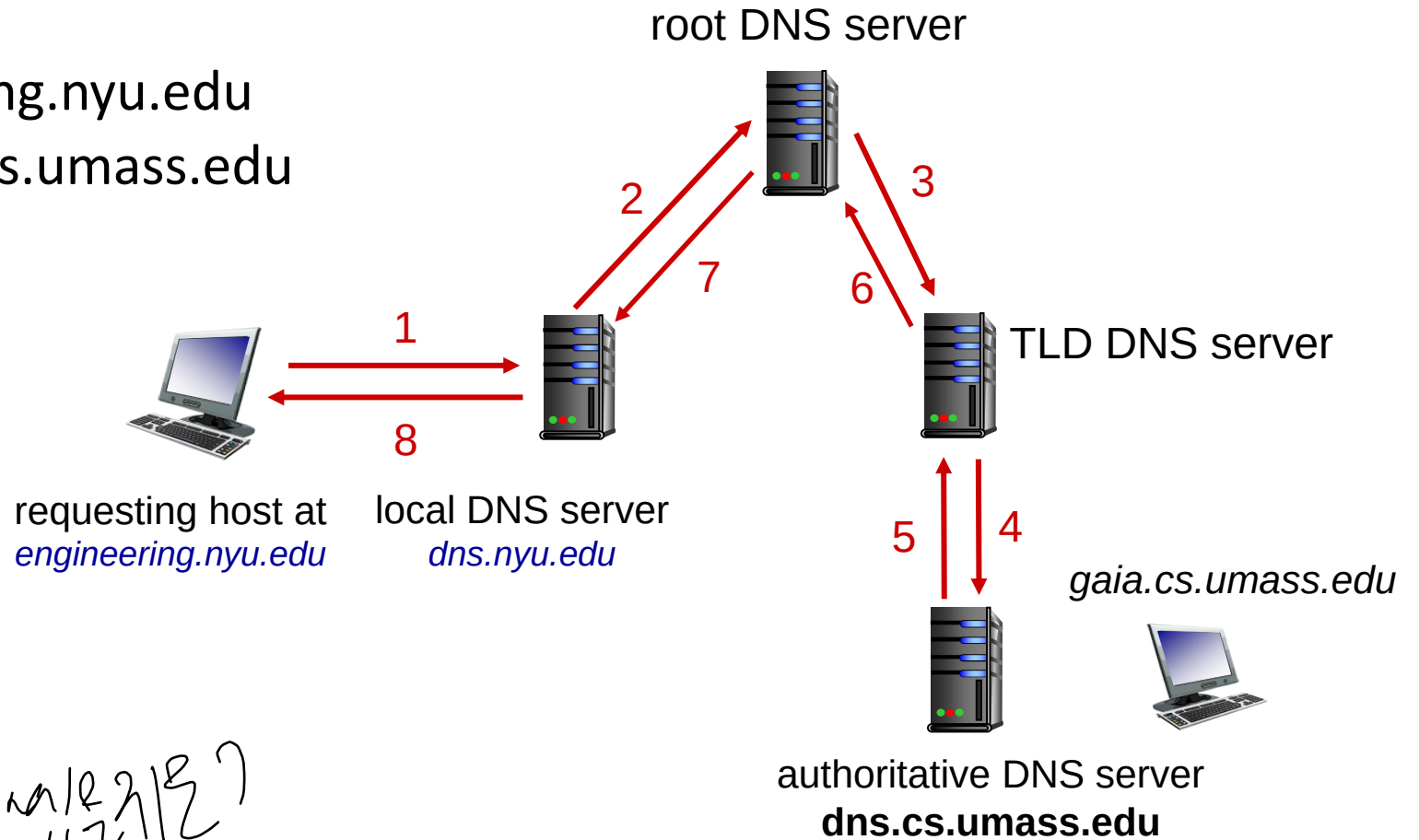


DNS name resolution: recursive query

Example: host at `engineering.nyu.edu` wants IP address for `gaia.cs.umass.edu`

Recursive query:

- Puts burden of name resolution on contacted name server
- Heavy load at upper levels of hierarchy? $\Rightarrow$ *1/2 2/2 3/2*



Caching DNS Information

- Once (any) name server learns mapping, it *caches* mapping, and *immediately* returns a cached mapping in response to a query
 - Caching improves response time
 - Cache entries timeout (disappear) after some time (TTL)
 - TLD servers typically cached in local name servers
- Cached entries may be *out-of-date*
 - If named host changes IP address, may not be known Internet-wide until all TTLs expire!
 - *Best-effort name-to-address translation!*

DNS records

DNS: distributed database storing resource records (RR)

RR format: (name, value, type, ttl)

type=A

- name is hostname
- value is IP address

type=NS

- name server
- name is domain (e.g., foo.com)
 - value is hostname of authoritative name server for this domain

type=CNAME

- name is alias name for some “canonical” (the real) name
- www.ibm.com is really servereast.backup2.ibm.com
- value is canonical name

type=MX

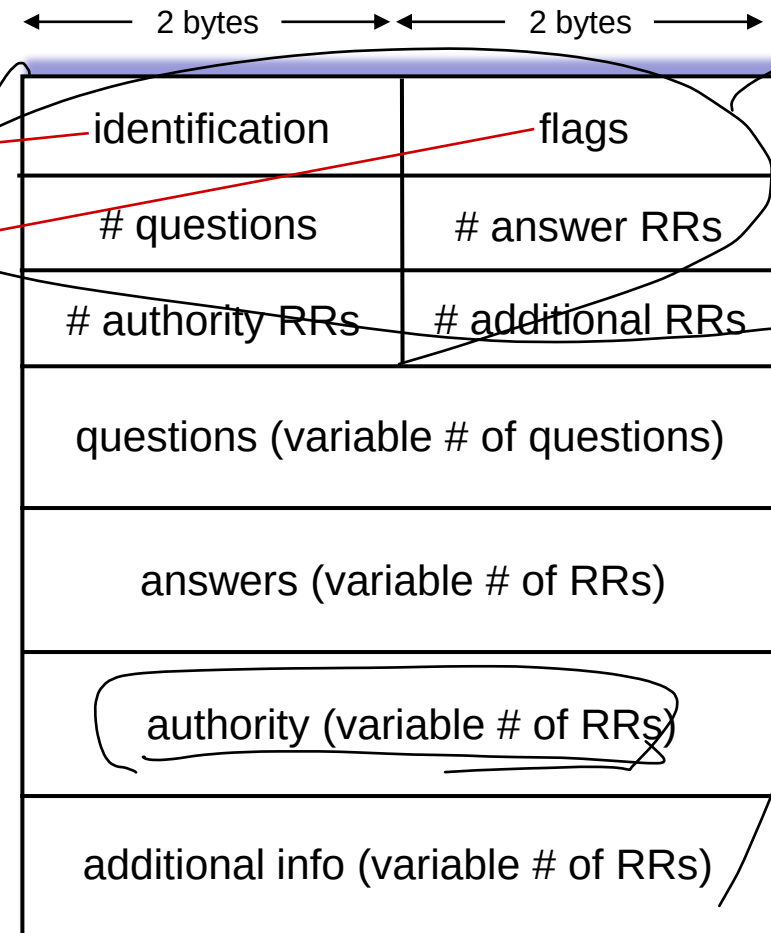
- value is name of SMTP mail server associated with name

DNS protocol messages

DNS *query* and *reply* messages, both have same *format*:

Message header:

- **Identification:** 16 bit # for query, reply to query uses same #
- **Flags:**
 - query or reply
 - recursion desired
 - recursion available
 - reply is authoritative



DNS protocol messages

DNS *query* and *reply* messages, both have same *format*:

← 2 bytes → ← 2 bytes →

identification	flags
# questions	# answer RRs
# authority RRs	# additional RRs
questions (variable # of questions)	
answers (variable # of RRs)	
authority (variable # of RRs)	
additional info (variable # of RRs)	

name, type fields for a query

RRs in response to query

records for authoritative servers

additional “helpful” info that may be used

Getting your info into the DNS

Example: new startup “Network Utopia”

- Register name networkutopia.com at *DNS registrar* (e.g., Network Solutions)
 - Provide names, IP addresses of authoritative name server (primary and secondary)
 - Registrar inserts NS, A RRs into .com TLD server:
$$\left(\begin{array}{l} (\text{networkutopia.com}, \text{dns1.networkutopia.com}, \text{NS}) \\ (\text{dns1.networkutopia.com}, 212.212.212.1, \text{A}) \end{array} \right)$$
- Create authoritative server locally with IP address 212.212.212.1
 - type A record for www.networkutopia.com
 - type MX record for networkutopia.com

DNS security

DDoS attacks

- Bombard root servers with traffic
 - Not successful to date
 - Traffic filtering
 - Local DNS servers cache IPs of TLD servers, allowing root server bypass
- Bombard TLD servers
 - potentially more dangerous

Spoofing attacks

- Intercept DNS queries, returning bogus replies
 - DNS cache poisoning
 - RFC 4033: DNSSEC authentication services

